РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ Факультет физико-математических и естественных наук

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2.

дисциплина: Архитектура компьютера

Студент: Исаев Р К.

Группа: НКАбд-01-24

МОСКВА

2025 г.

Цель работы

Цель данной лабораторной работы — научиться использовать систему контроля версий Git. В процессе работы изучались базовые команды, настройка SSH-доступа и организация файлов проекта в виде структурированных папок.

Описание выполнения работы

1. Создание репозитория и настройка Git:

Я создал локальный репозиторий с помощью команды git init.

Подключился к удалённому репозиторию на GitHub, настроив SSH-ключ с помощью команды ssh-keygen. Созданный ключ добавил в настройки GitHub.

2. Структура каталога:

Для удобства работы файлы были размещены в папках:

labs/lab01/report — отчёты первой лабораторной работы.

labs/lab02/report — отчёты текущей лабораторной работы.

3. Добавление файлов:•

В папку lab01/report добавлен файл lab1.pdf, содержащий отчёт по первой работе.

В папку lab02/report добавлен файл report.md для описания второй работы.

Файлы добавлены в репозиторий с помощью команды git add.

4. Работа с Git:

Я зафиксировал изменения с помощью команды git commit.

Отправил данные в удалённый репозиторий командой git push.

Использовал команду git status, чтобы следить за состоянием репозитория.

5. Исправление структуры:

Переместил файлы в нужные директории с помощью команды mv.

Удалил лишние папки командой rmdir.

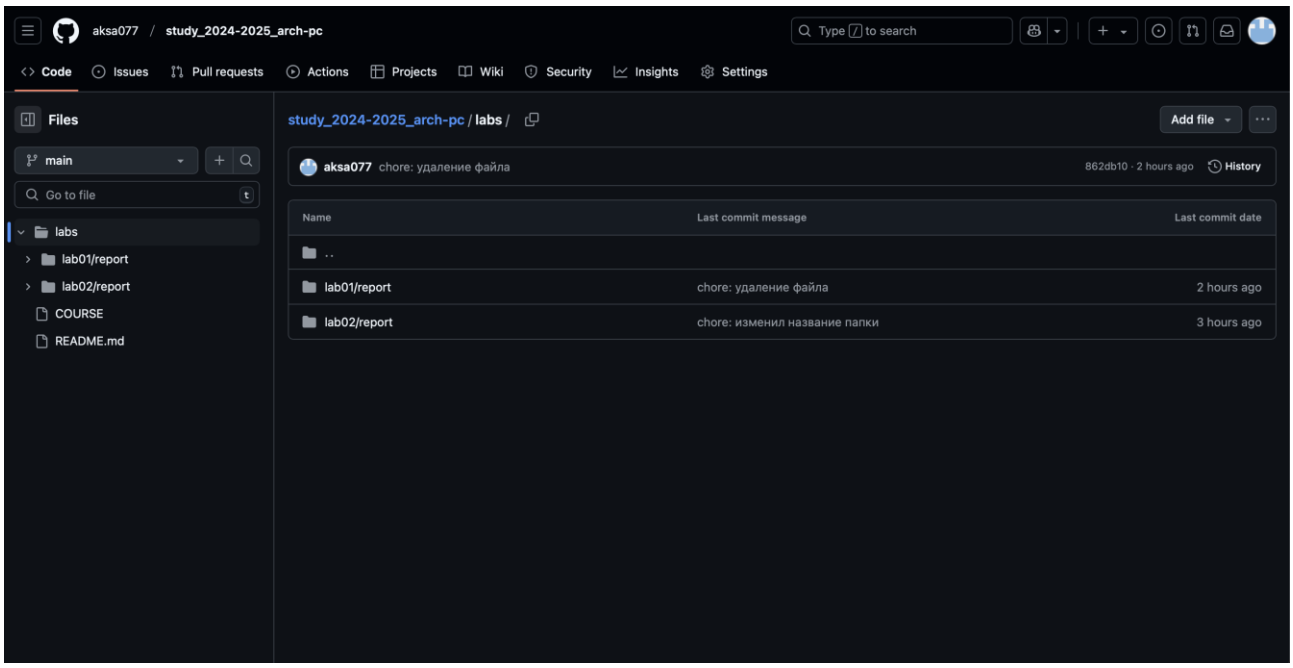
Проверил итоговую структуру с помощью команды ls -R.

```
study_2024-2025_arch-pc — -zsh — 108×31

labs/lab02:
lab2    report

labs/lab02/lab2:

labs/lab02/report:
report.md
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc % killall
usage: killall [-delmsqvz] [-help] [-I]
               [-u user] [-t tty] [-c cmd] [-SIGNAL] [cmd]...
At least one option or argument to specify processes must be given.
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc % clear
zsh: command not found: clear
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc % ls -R labs
lab01   lab02

labs/lab01:
report

labs/lab01/report:
lab1.pdf

labs/lab02:
lab2    report

labs/lab02/lab2:

labs/lab02/report:
report.md
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc %
```

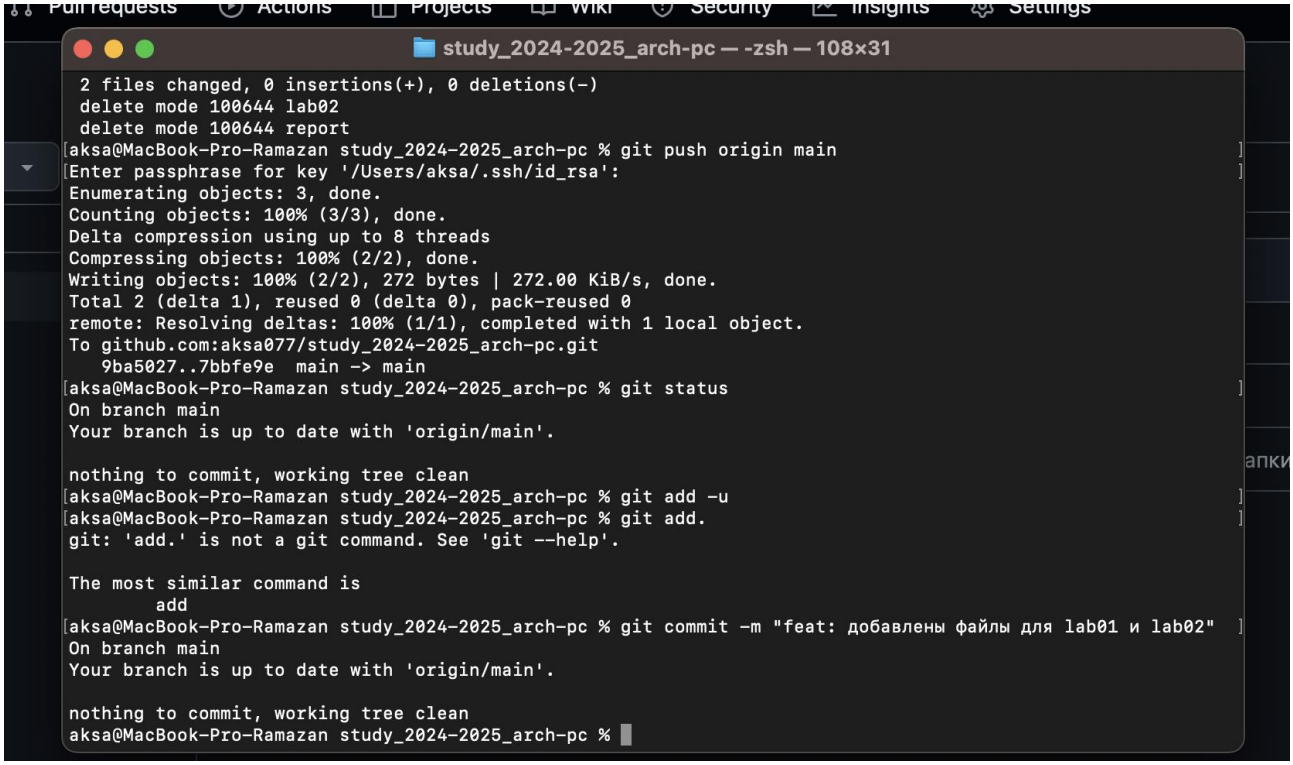
Команда ls -R labs

```
study_2024-2025_arch-pc — -zsh — 108×31

Everything up-to-date
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc % git add -u
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc % git commit -m "chore: удалены ненужные файлы lab02 и repo
rt"
[main 7bbfe9e] chore: удалены ненужные файлы lab02 и report
 2 files changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 lab02
 delete mode 100644 report
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc % git push origin main
Enter passphrase for key '/Users/aksa/.ssh/id_rsa':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 272 bytes | 272.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:aksa077/study_2024-2025_arch-pc.git
   9ba5027..7bbfe9e  main -> main
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc % git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc % git add -u
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc % git add.
git: 'add.' is not a git command. See 'git --help'.

The most similar command is
        add
aksa@MacBook-Pro-Ramazan study_2024-2025_arch-pc %
```

Команда git status



Репозиторий GitHub



Команда git add.

Выводы

Я научился работать с системой контроля версий Git: создавать репозитории, фиксировать изменения, отправлять их в удалённый репозиторий, а также правильно организовывать файлы в проекте. Настройка SSH-доступа упростила работу и сделала её безопасной. Эти знания помогут быстрее и удобнее выполнять задачи в будущих проектах.

Контрольные вопросы для самопроверки

1. Что такое системы контроля версий (VCS) и для чего они нужны?

Системы контроля версий (VCS) — это инструменты, которые помогают отслеживать изменения в коде или файлах. Они сохраняют разные версии проекта, чтобы можно было вернуться к любой из них, если что-то пошло не так.

Они нужны для: хранения всех версий проекта, совместной работы над одним проектом, восстановления удалённых или повреждённых данных.

2. Понятия VCS: хранилище, commit, история, рабочая копия

- **Хранилище** — это место, где хранятся все файлы проекта и их изменения (например, репозиторий в Git).
- **Commit** — это сохранение изменений в хранилище. Каждое сохранение имеет своё описание и метку времени.
- **История** — это список всех коммитов. Она показывает, кто и какие изменения внёс в проект.
- **Рабочая копия** — это версия проекта, над которой сейчас работает пользователь. Она может отличаться от того, что хранится в хранилище.

Отношения между ними: вы работаете с **рабочей копией**, делаете изменения и сохраняете их с помощью **commit**. Все сохранения попадают в **историю**, которая хранится в **хранилище**.

3. Чем отличаются централизованные и децентрализованные VCS?

- **Централизованные VCS** (например, SVN): есть один главный сервер, и все изменения сохраняются на нём. Если сервер недоступен, работать нельзя.
- **Децентрализованные VCS** (например, Git): каждый разработчик имеет полную копию проекта на своём компьютере. Даже если интернет пропадёт, можно работать локально.

4. Действия с VCS при единоличной работе с хранилищем

Если работаешь один:

1. Создаёшь хранилище (git init).
2. Добавляешь файлы (git add).
3. Сохраняешь изменения (git commit -m "Описание").

4. При необходимости подключаешь удалённое хранилище (git remote add origin <URL>).
5. Отправляешь изменения на удалённый сервер (git push).

5. Порядок работы с общим хранилищем VCS

Если работаешь в команде:

1. Клонируешь репозиторий (git clone <URL>).
2. Вносишь изменения в рабочую копию.
3. Добавляешь изменения (git add).
4. Сохраняешь изменения (git commit -m "Описание").
5. Синхронизируешься с командой:

- Забираешь изменения других (git pull).
- Отправляешь свои изменения (git push).

6. Если возникают конфликты, решаешь их вручную.

6. Основные задачи, решаемые git

- Отслеживание изменений в проекте.
- Работа с несколькими версиями проекта.
- Совместная работа с командой.
- Синхронизация локальных и удалённых изменений.
- Восстановление старых версий файлов.

7. Основные команды git

- git init — создание нового репозитория.
- git clone — копирование репозитория с сервера.
- git add — добавление изменений в индекс (подготовка к коммиту).
- git commit — сохранение изменений в репозитории.
- git push — отправка изменений на сервер.

- git pull — получение изменений с сервера.
- git status — проверка текущего состояния репозитория.
- git log — просмотр истории изменений.

8. Примеры работы с локальным и удалённым репозиториями

- **Локальный репозиторий:**

1. Создаём репозиторий: git init.
2. Добавляем файлы: git add ..
3. Сохраняем изменения: git commit -m "Добавил файлы".

- **Удалённый репозиторий:**

1. Подключаем удалённое хранилище: git remote add origin <URL>.
2. Отправляем изменения: git push origin main.
3. Забираем изменения: git pull origin main.